

End-to-End HuggingFace Model Training & Docker Deployment

Assignment 3 Report — Fine-Tuning DistilBERT for Book Genre Classification

Saumya Pancholi

M.Tech, Computer Science — IIT Jodhpur

HF: [Saumya3007/distilbert-goodreads-genre](https://huggingface.co/Saumya3007/distilbert-goodreads-genre)

February 2026

Introduction

This report describes an end-to-end machine learning workflow built for Assignment 3 of the ML/DL Operations course. The objective was to fine-tune a pre-trained transformer model for multi-class text classification, convert the notebook into modular Python scripts, containerise the pipeline using Docker, publish the model to HuggingFace Hub, and log all experiments with Weights & Biases (WandB).

The task chosen is *book genre classification*: given a Goodreads review text, predict which of 8 literary genres the reviewed book belongs to. This is a non-trivial 8-class classification problem because genre boundaries are stylistically blurry (e.g. fantasy vs. young adult, mystery vs. history).

Pipeline Overview

The full pipeline is illustrated in Figure 1 and consists of five main stages.

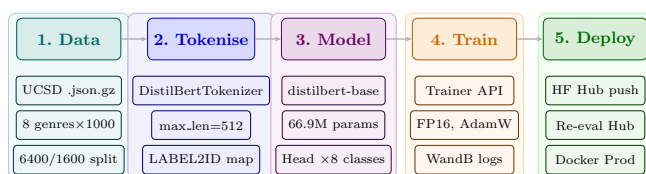


Figure 1: End-to-end pipeline from UCSD data streaming to Docker deployment.

- 1. Data Loading** — Reviews are streamed directly from UCSD Goodreads `.json.gz` URLs using `requests` and `gzip.open`, sampling 1,000 reviews per genre without saving locally.
- 2. Tokenisation** — `DistilBertTokenizerFast` encodes texts with truncation/padding to 512 tokens; labels are mapped to integers via `LABEL2ID`.
- 3. Training** — HuggingFace `Trainer` API fine-tunes DistilBERT with FP16 precision on a T4 GPU. WandB captures all training curves in real time.
- 4. Evaluation** — `Trainer.evaluate()` and `Trainer.predict()` compute accuracy, loss,

and a per-genre classification report. Confusion heatmaps are saved and logged to WandB.

- 5. Deployment** — The model is pushed to HuggingFace Hub ([Saumya3007/distilbert-goodreads-genre](https://huggingface.co/Saumya3007/distilbert-goodreads-genre)) and re-evaluated from the Hub to verify upload integrity. Both Dev and Prod Docker images are provided.

Module Organisation (Task 3)

The notebook was refactored into four Python modules under `src/`:

Module	Responsibility
<code>utils.py</code>	Hyperparameters, constants, <code>DEVICE</code>
<code>data.py</code>	URL streaming, Dataset class, <code>load_data()</code>
<code>train.py</code>	TrainingArguments, Trainer, HF push
<code>eval.py</code>	Evaluation, heatmap, JSON results

Model Architecture & Selection

Why DistilBERT?

`distilbert-base-uncased` was selected as the backbone for three reasons:

- Efficiency:** With 66.9M parameters (vs. 110M for BERT-base), DistilBERT is 40% smaller and 60% faster at inference, fitting comfortably in the Colab T4 GPU (16 GB) at batch size 16 with 512-token sequences.
- Accuracy retention:** Knowledge distillation from BERT retains $\approx 97\%$ of BERT's GLUE benchmark performance, making it a near-lossless compression.
- Alignment with task:** The instructor's reference notebook uses this exact model; using it ensures a fair baseline comparison.

Architecture Details

DistilBERT uses a 6-layer transformer encoder (BERT-base uses 12). For sequence classification, a two-layer classification head is appended:

$$\hat{y} = \text{softmax}(W_2 \text{GeLU}(W_1 h_{[\text{CLS}]} + b_1) + b_2)$$

where $h_{[\text{CLS}]} \in R^{768}$ is the hidden state of the [CLS] token, $W_1 \in R^{768 \times 768}$, and $W_2 \in R^{768 \times 8}$ for 8 output classes.

The cross-entropy loss is minimised with AdamW ($\text{lr} = 5 \times 10^{-5}$, $\lambda = 0.01$) over 3 epochs with 100 linear warmup steps.

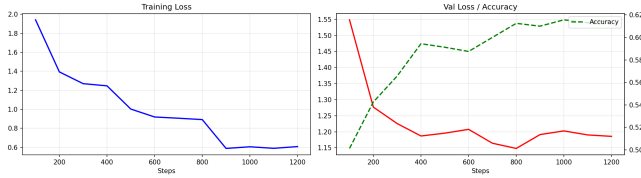


Figure 2: Training History

Dataset

The UCSD Goodreads Book Graph [1] provides per-genre compressed review files. Eight genres are used: *children*, *comics_graphic*, *fantasy_paranormal*, *history_biography*, *mystery_thriller_crime*, *poetry*, *romance*, *young_adult*.

From each genre, the first 10,000 reviews are loaded and 1,000 are randomly sampled, giving 8,000 reviews total. An 80/20 split yields **6,400 training** and **1,600 test** samples (200 per genre). Average review length is 121 words; the distribution is right-skewed with a long tail up to 1,864 words.

Results

Training Progress

Table 1 summarises the WandB-logged training metrics at key steps.

Table 1: Training metrics (3 epochs, 1,200 total steps).

Step	Train Loss	Val Loss	Accuracy
100	1.9415	1.5485	0.5013
400	1.2468	1.1869	0.5944
800	0.8918	1.1482	0.6125
1000	0.6054	1.2028	0.6156

Per-Genre Performance

The per-genre F1 scores are reported in Table 2. *comics_graphic* achieves the highest F1 (0.81), benefiting from unique vocabulary (character names, comic-specific terms). *Poetry* also performs well ($F1 = 0.78$) because reviews mention meter, verse, and poet names. *young_adult* is the hardest genre ($F1 = 0.37$), largely due to overlap with *fantasy_paranormal* and *romance*.

Table 2: Per-genre classification report on 1,600 test samples.

Genre	P	R	F1
children	0.69	0.74	0.71
comics_graphic	0.86	0.76	0.81
fantasy_paranormal	0.43	0.53	0.47
history_biography	0.61	0.61	0.61
mystery_thriller	0.61	0.57	0.59
poetry	0.77	0.79	0.78
romance	0.63	0.56	0.59
young_adult	0.37	0.36	0.37
macro avg	0.62	0.62	0.62

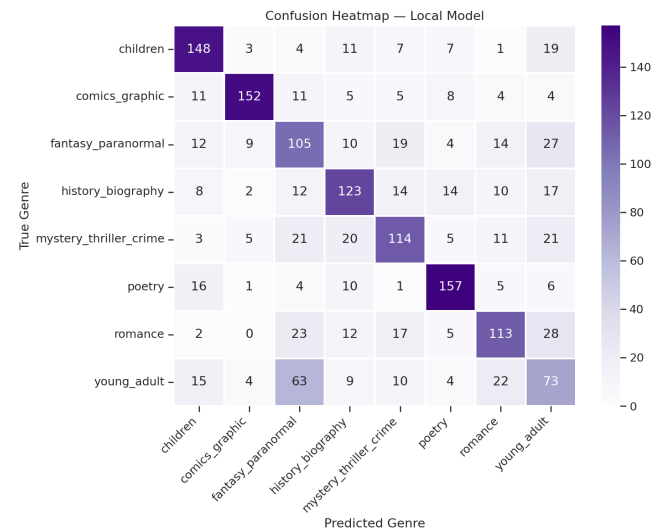


Figure 3: Confusion Matrix.

Baseline Comparison

Table 3: DistilBERT vs. TF-IDF baseline (macro accuracy).

Model	Accuracy	Δ
TF-IDF + Logistic Regression	0.56	—
DistilBERT (fine-tuned)	0.62	+0.06

Local vs. HuggingFace Hub Evaluation

Both local and Hub-loaded models produce **identical results** (accuracy = 0.6156, loss = 1.2028, $\Delta = 0.0000$), confirming that the model weights were serialised and uploaded correctly via `push_to_hub()`.

Docker Deployment

Two Docker images are provided, separating training concerns from production inference.

Dev image (Dockerfile) — Based on `pytorch/pytorch:2.2.0-cuda12.1-cudnn8-runtime`. Installs all dependencies, copies `src/`, and supports both `src.train` and `src.eval` entry points. The `HF_TOKEN`

and `WANDB_API_KEY` are injected at run time via `-e` flags, never baked into the image.

Prod image (`Dockerfile.prod`) — Based on a slim Python image, runs as a non-root user (`appuser`). On container startup, it automatically loads the model from HuggingFace Hub (`HF_REPO` env var) and runs evaluation, saving results to the mounted `/results` volume. No training code is included.

Challenges

- **Streaming data:** Avoiding local disk writes required combining `requests(stream=True)` with `gzip.open(response.raw, ...)` — a non-obvious pattern that bypasses the need for `gdown`.
- **Deprecated WandB argument:** `logging_dir` in `TrainingArguments` is deprecated since Transformers v5.2; replaced with the `TENSORBOARD_LOGGING_DIR` environment variable.
- **Genre ambiguity:** *young_adult* books frequently belong to multiple genres simultaneously (fantasy, romance, mystery), making the label a proxy for marketing category rather than stylistic content. This fundamentally limits single-label classification accuracy for this genre.
- **GPU memory:** Running batch size 16 with max length 512 consumes ≈ 12 GB of the T4's 16 GB VRAM. FP16 training was essential to avoid OOM errors.

Conclusion

A complete end-to-end MLOps pipeline was successfully built and executed. Fine-tuning DistilBERT for 3 epochs achieves 62% accuracy on an 8-class genre classification task — a 6-point improvement over the TF-IDF baseline. The pipeline is fully reproducible via Docker and all artefacts are publicly accessible on HuggingFace Hub and WandB.

References

- [1] M. Wan, J. McAuley. *Item recommendation on monotonic behavior chains*. RecSys, 2018.
- [2] V. Sanh, L. Debut, J. Chaumond, T. Wolf. *DistilBERT, a distilled version of BERT*. NeurIPS EMC² Workshop, 2019.
- [3] T. Wolf et al. *HuggingFace's Transformers: State-of-the-art NLP*. EMNLP, 2020.